

DALI · DESIGN ENGINEERING 26F

Design Engineering

Session 3: Working With an Agent, For Real

Petros Taxiarchis · October 8, 2026

Room B05

TODAY, AT A GLANCE

The whole run of show

0:00	Welcome back + roadmap
0:02	How an agent actually works
0:06	Context: what the agent can see
0:11	Permissions & safety
0:14	Writing a spec the agent can follow
0:18	Pair exercise: rewrite a bad prompt
0:21	Skills, a quick tour
0:24	Live demo: ship a feature with an agent
0:33	Reviewing what the agent wrote
0:36	Stretch
0:38	Figure it out: the skill you can't outsource
0:42	Live demo: figure out a real bug
0:46	Lab: three bugs, no hints
0:57	Show & tell
0:59	Wrap-up

WELCOME BACK

0:00 · 2 min

LECTURE

Last time, this time

- Session 1: git and your first agent. Session 2: the stack menu and your ADR
- Today: how to actually **work** with an agent. What it sees, how to ask it, how to check what it did, and what to do when it (or you) gets stuck
- Claude Code is already set up from Session 1. **No installs, no config today**
- Using your Dartmouth Chat key instead? Setup is in the appendix slides at the end, and in section 0 of the cheatsheet
- Everything you might want to copy is in one file in the repo: `docs/session-3-cheatsheet.md`. Don't copy from the slides

AGENTS, FOR REAL

0:02 · 4 min

LECTURE

What an agent actually is

A language model that can do more than talk: it can use **tools**, and it keeps going until it decides it's done.

- Tools are things like: read a file, edit a file, run a terminal command, search the code
- It isn't "thinking about your project". It's reading text, producing text, and acting on the result
- It has no memory of you. Everything it knows about your task is **whatever is in front of it right now**

AGENTS, FOR REAL

0:02 · 4 min

LECTURE

The loop

1 · Read



2 · Plan



3 · Act



4 · Check



- **Read:** opens files, greps, looks at your project
- **Act:** edits files, runs commands
- **Check:** reads the output of what it just did (a test, an error, a server log) and decides what's next
- The Check step is the whole difference between "vibe coding" and agentic coding. If the agent has *no way to check*, it guesses. If you give it a test or a command to run, it can correct itself

CONTEXT

0:06 · 5 min

LECTURE

Context: the agent's entire world

Every conversation has a **context window**: a fixed-size pile of text the model can see at once. Your messages, the files it opened, the output of every command, all in the pile.

- If it's not in the pile, the agent doesn't know it. Not your other tab, not yesterday's session, not what you "obviously meant"
- The pile fills up. A long, messy session full of dead ends and giant outputs makes the agent **slower, more expensive, and worse**: it loses track of early instructions
- "Context engineering" is just the job of putting the right things in the pile, and keeping the wrong things out

CONTEXT

0:06 · 5 min

LECTURE

What goes in the pile

- Your prompts
- Every file the agent reads
- Every command's output
- `CLAUDE.md` : a file of standing instructions, loaded at the start of every session
- Skills it decides to load (more in a few minutes)

What's NOT in it

- The reason you want the feature
- Your last session
- The convention your team uses, unless it's written down
- Anything you "mentioned earlier" that has scrolled out or been summarized away

Rule: if you'd have to explain it to a new teammate, it has to be written down somewhere the agent reads.

CONTEXT

0:06 · 5 min

LECTURE

Managing the pile

```
/clear      # new task? wipe it and start clean. The most underused command  
/compact    # same task, but the conversation is huge: summarize it  
/context     # show me what's actually filling the window  
@routes.js  # point the agent at the exact file instead of making it hunt
```

- **One task per session.** Finished the feature? `/clear` before starting the next thing
- **Point, don't describe.** "Look at `@routes.js`" beats "somewhere in the backend there's a file"
- **Write down what repeats.** Commands, layout, rules: that's what `CLAUDE.md` is for. A sample is in the cheatsheet. It's an example only: this repo doesn't have a `CLAUDE.md` yet

PERMISSIONS & SAFETY

0:11 · 3 min

LECTURE

How much do you let it do on its own?

Tap `Shift+Tab` to cycle through modes.

- **Default:** asks before editing a file or running a command. Slow, but you see everything
- **Accept edits:** edits files freely, still asks before running commands. Good once you trust the plan
- **Plan mode:** read-only. It can explore and propose, but can't change anything. Use it for every task that touches more than one file
- There's also a "skip all permission checks" flag. **Don't.** Not on your laptop, not on a project that has real data

PERMISSIONS & SAFETY

0:11 · 3 min

LECTURE

Four habits that make agents safe

1. **Own branch.** `git switch -c session-3-yourname`. Never work on master
2. **Checkpoint before every run.** `git add -A && git commit -m "checkpoint"`. If it goes sideways, you're one command from before
3. **Read the diff after every run.** `git diff`. If you can't explain a change, don't keep it
4. **Keep secrets and real data away.** No `.env` contents in prompts, no production database connection, ever

Session 1's git is what makes everything else today low-stakes.

SPECS

0:14 · 4 min

LECTURE

A prompt is a spec for a very literal reader

Vague

- "Add a popular clubs endpoint"

What does "popular" mean? Members? Recent joins? What's the response shape? Does it need a test? The agent will pick answers for you, and won't tell you it did.

Specific

- Ranked by member count, biggest first
- `?limit=3`, default 5
- Same pattern as `/dining/today`
- A smoke test, and `npm test` passes

The shape of a good prompt

Goal: one sentence. What exists when this is done?

Where: @files to look at first

Pattern: "follow how X already does it"

Done when:

- a request I can run, and what it returns
- a test that passes

Don't: add dependencies / touch other files

Plan first. Don't write code until I approve.

- **"Done when" is the important one.** It's the Check step from the loop, written by you in advance
- **"Follow the pattern"** beats any description of style. Agents are great at imitating code that's already there
- The full template is in the cheatsheet, ready to copy

PAIR EXERCISE

0:18 · 3 min

HANDS-ON

Your turn: rewrite a bad prompt

Turn to the person next to you. Pick **one**, rewrite it using Goal / Where / Pattern / Done when / Don't. No agent needed, just write it.

1. "Make the dining menu better"
2. "Add search to the students endpoint"
3. "Fix the clubs thing"

Hint for #3: you'll have to *decide* what's broken first. That's the point.

3 minutes. We'll hear two.

SKILLS

0:21 · 3 min

LECTURE

Skills: instructions the agent loads when it needs them

A skill is a folder with one file, `SKILL.md`. The agent reads only the one-line **description** up front, and loads the full body when the task matches.

```
---  
name: api-route  
description: Use when adding or changing an endpoint in this API.  
---  
1. Add the handler in routes.js, next to the closest route  
2. Add a line to computed-routes.js  
3. Add a test to smoke-test.js, run npm test
```

- This one is an example, not a file in the repo. Why not put it all in `CLAUDE.md`? That loads *every* session. A skill only costs context when it's relevant
- A skill file is a design system for an agent: your conventions, written once, applied every time

SKILLS

0:21 · 3 min

LIVE DEMO

A tour of one: my router skill

Demo only. You're not installing anything today.

- I keep a skill called `stack-router`: a table that says "this kind of task → use this other skill"
- UI work routes to a design skill. Mobile work routes to Expo skills. Small fixes route to a quick-fix workflow
- Skills **stack**: one routes to another, and each only loads when the task needs it
- Open it on screen, scroll the table, and point out the rules in it. The "anti-rules" matter most: they stop the agent from over-doing it

LIVE DEMO

0:24 · 9 min

LIVE DEMO

Ship a feature with an agent

Watch the process, not just the result. Task: **add** `GET /clubs/popular` to the student API.

1. New branch, clean `git status`, checkpoint commit
2. Start in **plan mode**, paste the spec
3. **Read the plan** before approving. Is it touching the files I expect?
4. Approve. Let it run. Watch it run `npm test`
5. `git diff`: read every line
6. Curl it myself. Commit

LIVE DEMO

0:24 · 9 min

LIVE DEMO

The spec we paste

Goal: add GET /clubs/popular: clubs ranked by member count.

Where: @routes.js (see /dining/today), @computed-routes.js, @smoke-test.js

Done when:

- GET /clubs/popular?limit=3 returns 3 clubs, each with member_count, biggest first. limit defaults to 5; "abc" falls back to 5
- listed in computed-routes.js so it appears in /openapi.json
- new smoke test, npm test passes

Don't add dependencies or touch data.js.

Make a plan first.

Expect a snag: `/clubs/popular` may get swallowed by `/clubs/:id` and answer "clubs popular not found". Narrate how the test catches it, and how you'd tell the agent what you saw.

REVIEWING

0:33 · 3 min

LECTURE

What agents get wrong (the "tells")

Scope creep

- Edited files you didn't mention, "tidied up" code that was fine

Surprise dependencies

- New package in `package.json` for something 5 lines could do

Swallowed errors

- An empty `catch` or a fallback that hides the failure

Tests that can't fail

- Asserts something always true, or mocks the very thing being tested

And one more: the confident summary. "All done, everything works" is a claim. **Run it yourself.**

REVIEWING

0:33 · 3 min

LECTURE

Three questions for every diff

1. **Did it do what I asked?** Walk the "done when" list. Run each item
2. **Did it do more than I asked?** `git diff --stat`: any file you didn't expect?
3. **Could the tests fail?** Break the code on purpose. Does the test notice?

You can also ask a second agent, in a fresh session, to review the diff: "list problems only, don't fix". A fresh session has no loyalty to the code it didn't write. (Prompt in the cheatsheet.)

BREAK

0:36 · 2 min

BREAK

Stretch

Two minutes. Stand up, refill water. When we're back: what to do when it doesn't work.

FIGURE IT OUT

0:38 · 4 min

LECTURE

**The agent will get stuck.
So will you.
The skill is what you do next.**

- Agents are good at routine code and bad at a specific kind of problem: when the real cause isn't where the error points
- "It doesn't work, fix it" gives the agent nothing to work with. It guesses. You get a confident, wrong change
- People who get unstuck fast aren't smarter. They run a loop, in order

FIGURE IT OUT

0:38 · 4 min

LECTURE

The loop

1. **Read the whole error.** First line says what, next lines say where. Find the first line that's *your* code
2. **Reproduce it.** What exact input makes it fail, every time?
3. **Shrink it.** Smallest input that still fails
4. **Look at the real values.** `console.log` right before the failure. Is it what you assumed?
5. **Say expected vs. actual.** The gap is the bug
6. **Read the docs** for the one function involved
7. **Then ask the agent, with evidence.** Ask it to *explain* first

FIGURE IT OUT

0:38 · 4 min

LECTURE

Asking for help: bad vs. good

Bad

```
it doesn't work, fix it
```

No error, no input, no expectation. The agent has to guess all three.

Good

```
GET /students?year=senior  
returns [] but I expected  
seniors. year=Senior works.  
Explain why. Don't change  
any files yet.
```

Input, actual, expected, what you already know, and a constraint.

FIGURE IT OUT

0:42 · 4 min

LIVE DEMO

Live: figure out a real bug

This one is in our API right now. `GET /students?year=senior` returns `[]`.

1. **Reproduce.** curl it. Then try `year=Senior`. That works. First clue
2. **Look at the real values.** curl one student. The value is `"Senior"`, capital S
3. **Expected vs. actual.** I expected case not to matter. The filter compares exact strings
4. **Now ask the agent**, with all of that, in "explain, don't fix" mode. Find the line in `routes.js`
5. **Decide.** Is this a bug, or a design choice? Fix it, or document it? That call is yours, not the agent's

LAB

0:46 · 11 min

HANDS-ON

Lab: three bugs, no hints

```
git switch -c session-3-yourname  
npm run figure-it-out      # 3 tests fail. Make them pass
```

1. Open `figure-it-out/README.md`. Three bugs, three *different kinds*: a crash, a wrong number, a wrong order
2. First message to the agent, every time: **"explain why this fails, don't change anything yet"**
3. Don't edit the test file. The tests are right
4. Stuck 5 minutes? Switch tools: print values, read MDN, shrink the input, or `/clear` and describe it better
5. Commit after each green test

Done early? One line per bug in `NOTES.md`: what actually told you the cause. Then the stretch problem.

LAB

0:46 · 11 min

HANDS-ON

While you work

- Hands up if you're stuck for more than 5 minutes. Say what you've tried, not just "it's broken"
- Check `git diff` after the agent touches anything. Did it edit `lib.test.js`? Undo that: `git restore figure-it-out/lib.test.js`
- If you finish all three: stretch is `formatTerm("26F")` → `"Fall 2026"`. Write the test for garbage input first
- No homework. Whatever isn't done stays undone, and that's fine

SHOW & TELL

0:57 · 2 min

SOCIAL

Show & tell

- A couple of volunteers. One sentence: **which bug took longest, and what finally told you the cause?**
- "The error message, I just hadn't read it" is a completely valid answer

WRAP-UP

0:59 · 1 min

BREAK

Next time

- Session 4: APIs, HTTP & Bruno. Calling the live API properly, and designing your own
- Keep `docs/session-3-cheatsheet.md`. You'll use the git and prompt sections all term
- No assigned homework

APPENDIX · REFERENCE

Not timed

REFERENCE

Claude Code with your Dartmouth Chat key

- Get the key: Dartmouth Chat → profile picture → Settings → Account → API Key. It starts with `sk-`
- It's a password. Never in a prompt, a slide, or a commit
- Put the settings in `.claude/settings.local.json` inside your project, not your global settings
- Base URL: `https://chat.dartmouth.edu/api` · model: `anthropic.claude-sonnet-4-6`
- The full JSON to copy is in section 0 of `docs/session-3-cheatsheet.md`
- Source: Dartmouth Research Computing, `rc.dartmouth.edu/ai/online-resources/connecting-ai-clients`

APPENDIX · REFERENCE

Not timed

REFERENCE

Where the key lives

macOS (Dartmouth's steps)

```
security add-generic-password \  
  -s "dartmouth-chat-api-key" \  
  -a ${USER} -w  
# paste the key, press return
```

- `apiKeyHelper` reads it back from the keychain

Linux (adapted, untested)

```
chmod 600 ~/.dartmouth-chat-key  
# apiKeyHelper:  
# cat ~/.dartmouth-chat-key
```

- Save the key in that file first (cheatsheet has the command)

Then run `claude` in the folder. `git status` should not show your settings file. Windows: ask in class.

Thank you!

Please stay for a moment, we're doing attendance in person.